FPGA-Accelerated Floating-Point Matrix Multiplication System on Zynq-7000

Implemented using Vitis HLS, Vivado, AXI4-Stream, and AXI DMA

Payal Mohnani
M.Tech VLSI Design and Nanoelectronics

DISCIPLINE OF ELECTRICAL ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY, INDORE

Date: April 20, 2026

Abstract

Matrix multiplication is a computationally intensive operation widely used in digital signal processing, machine learning, graphics, and scientific computing. In this project, a floating-point matrix multiplication accelerator was designed using Vitis HLS 2025.2 and implemented on the AMD/Xilinx Zybo Zynq-7010 FPGA platform. Due to the on-chip resource constraints of the XC7Z010 FPGA, a hybrid hardware-software orchestration strategy was employed to process large 1024×1024 matrices.

The hardware accelerator was designed for 32×32 sub-matrix tiles using AXI Stream interfaces and AXI DMA for high bandwidth data communication between the Processing System (PS) and the Programmable Logic (PL).

Optimisation techniques, including loop pipelining and array partitioning, were explored to improve performance while remaining within the resource limitations of the Zybo board. Three design solutions were evaluated and compared in terms of latency, resource utilisation, and timing performance.

A tiling computation technique was used to provide a full 1024x1024 matrix multiplication. Software partitions the 1024×1024 matrices into smaller 32×32 blocks, feeds them to the Hardware Accelerator sequentially to process, while partial results are accumulated in the system DDR3 memory. This approach demonstrates a scalable solution for efficiently performing large matrix multiplication with limited FPGA resources.

Table of Contents

List of Figures

List of Tables

1. Introduction

Matrix multiplication is one of the fundamental operations in many computational applications such as image processing, machine learning, digital signal processing, scientific simulations, and computer graphics. However, matrix multiplication involves a large number of multiply-and-accumulate operations, making it computationally expensive for software running on a general-purpose processor.

Field Programmable Gate Arrays (FPGAs) provide an efficient platform for accelerating such operations because they allow parallel execution of arithmetic computations in hardware. The Zynq-7000 architecture combines an ARM processing system with programmable logic on the same chip, making it suitable for hardware-software co-design. Computationally intensive tasks can be implemented in programmable logic, while the processor handles control, memory management, and software execution.

In this project, a floating-point matrix multiplication accelerator was developed using Vitis HLS. The accelerator performs matrix multiplication and communicates with the processing system through AXI4-Stream and AXI DMA interfaces. The design was implemented on the Zybo Zynq-7010 board operating at 100 MHz, using Vitis 2025.2 for HLS IP generation and software integration and Vivado 2025.2 for Hardware System Integration.

Due to the limited DSP, BRAM, and logic resources available on the Zybo board, directly implementing a 1024×1024 matrix multiplier entirely in hardware is not possible, as one 1024×1024 matrix with floating point numbers takes 4MB of space, exceeding the internal BRAM capacity of 260KB. Therefore, a hybrid computation approach using tiled computation was used. The large matrices were divided into smaller 32×32 blocks, which were processed sequentially by the hardware accelerator, while the ARM processor accumulated the intermediate results in software. This approach allowed large matrix multiplication to be supported without exceeding the hardware limitations of the target device.

2. Objectives

- To design a tiled floating-point matrix multiplication accelerator supporting 1024×1024 matrices using Vitis and Vivado.
- To implement the accelerator on the Zybo Zynq-7010 FPGA platform.
- To interface the hardware accelerator with the ARM Processing System using AXI4-Stream and AXI DMA protocols.
- To explore multiple HLS optimisation techniques to produce 3 distinct design solutions: a baseline version (no optimisation), a partially optimised version, and a fully optimised version.
- To compare these design solutions based on latency, timing performance (slack), and hardware resource utilisation. (BRAM, DSP, LUT).
- To develop software capable of performing 1024×1024 matrix multiplication using tiling, and partial result accumulation in DDR3 memory.
- To measure the total execution time for the process on software and hardware collectively.

3. System Architecture

The system architecture is based on the **Zynq-7000 All-Programmable SoC**, leveraging the **Processing System (PS)** and **Programmable Logic (PL)** domains. The ARM Cortex-A9 processor within the PS serves as the system orchestrator, responsible for software execution, memory management, DMA configuration, timer control, and interrupt handling. The PL hosts the hardware accelerator alongside the essential AXI infrastructure required for data movement and system control.

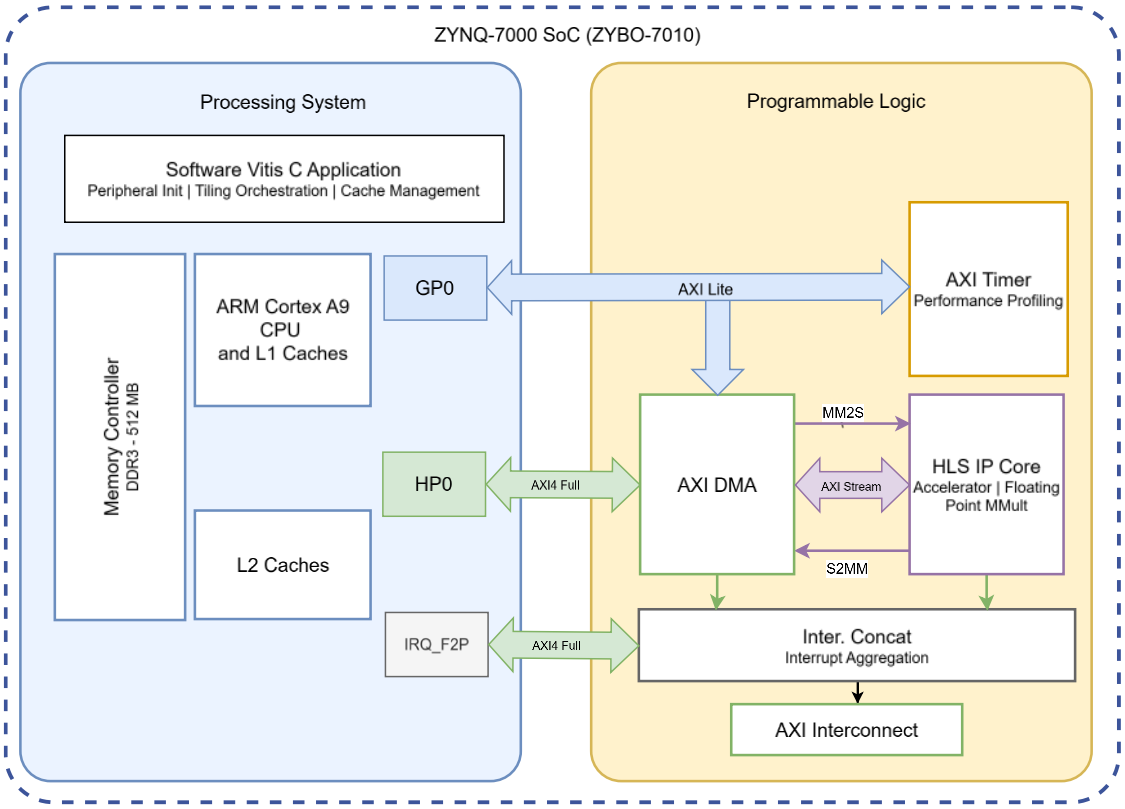

Figure 1 System Architecture for ZYNQ-7000 SoC

3.1. Hardware System Components

- **Zynq Processing System:** The primary control and memory management unit.
- **AXI DMA:** Facilitates high-speed data transfers between DDR memory and the hardware accelerator.
- **HLS Matrix Multiplication IP:** The custom kernel optimised for floating-point tile processing.
- **AXI Timer:** Used for high-precision performance profiling of both hardware and software implementations.
- **AXI Interconnect / SmartConnect:** Manages the routing of AXI transactions across the system.

- **Processor System Reset:** Manages the reset synchronisation for both PS and PL domains.
- **Interrupt Concat:** Aggregates interrupt signals from the DMA, timer, and HLS accelerator for the PS.

3.2. Data and Control Flow

The communication and execution flow is handled through dedicated AXI interfaces:

- **Control Plane (AXI4-Lite):** The PS GP0 (General Purpose) master port is used to access the HLS IP and DMA registers for configuration and control (Start/Stop/Idle). The **AXI-Lite** interface was used for control (Start/Done signals) via the **GP0** port
- **Data Plane (AXI4-Stream): AXI-Stream** was used for high-speed data movement via the **AXI DMA** connected to the **HP0** port. The HLS accelerator receives input matrices via AXI4-Stream. The **AXI DMA** acts as the data mover:
    - **MM2S (Memory-Mapped to Stream):** Transfers input tiles from DDR memory to the accelerator.
    - **S2MM (Stream to Memory-Mapped):** Transfers computed results from the accelerator back to DDR memory.
- **High-Speed Interconnect:** The HP0 (High-Performance) slave port is utilised for DMA transfers, providing the bandwidth necessary for efficient memory access between the DDR and PL.

3.3. Integration and Synchronisation

Interrupts are consolidated via the **Interrupt Concat block** and forwarded to the PS through the **IRQ_F2P** (Interrupt Request From PL) interface. To optimise bandwidth, the PS GP0 master port is utilised for low-latency AXI-Lite control transactions, while the **HP0 (High-Performance) port** is reserved for high-speed DMA data transfers between the DDR3 memory and the programmable logic. The entire system operates at a synchronised clock frequency of 100 MHz.

3.4. Memory Mapping and Address Space

The system utilises a memory-mapped architecture where the ARM processor interacts with PL-based peripherals through a defined 32-bit address space. The **AXI Interconnect** translates these addresses into control signals for the respective IP cores.

3.4.1. Peripheral Address Map

The base addresses for the key hardware components are configured in the Vivado Address Editor. These addresses are essential for the software driver initialisation in Vitis:

Table 1 Peripheral Address Map

Peripheral	Base Address	Range	Function
HLS Matrix IP	0x4000_0000	64 KB	Control registers (Start, Done, Idle)
AXI DMA	0x4040_0000	64 KB	DMA configuration and status (MM2S/S2MM)
AXI Timer	0x4280_0000	64 KB	High-resolution timing and interrupts

- **4.1.2 System Memory (DDR3) Allocation**

The external DDR3 memory (512 MB on the Zybo Z7-10) is shared between the Processing System and the DMA controller. To handle the 1024×1024 matrix operations, the memory is partitioned as follows:

- **PS Memory Space:** The lower portion of the DDR is reserved for the standalone application code, stack, and heap.
- **Data Buffers (.bss):** Large global arrays for Matrices A, B, and C are allocated in the .bss section of the DDR. Given that each 1024×1024 floating-point matrix requires **4 MB**, a total of **12 MB** of contiguous DDR space is reserved.
- **DMA Access:** The AXI DMA accesses these buffers via the **S_AXI_HP0** high-performance port, allowing it to bypass the CPU cache (if required) or utilise cache-coherent transfers.
- **4.1.3 Cache Coherency and the "Memory Wall"**

A critical aspect of the memory architecture is the management of the **L1/L2 Data Caches**. Since the ARM CPU writes the matrix data to the DDR through its cache, but the DMA reads directly from the physical DDR3 memory, the software must perform **Cache Flushing** (Xil_DCacheFlushRange) before a hardware operation. Conversely, it must perform **Cache Invalidation** (Xil_DCacheInvalidateRange) after the hardware writes the results back to memory to ensure the CPU reads the updated values rather than stale data from its cache.

4. HLS Design and Optimisation

4.1. Methodology and Design Entry

Matrix-Multiplication algorithm A×B = C is simple. There are 3 nested loops:

L1: iterates over the elements composing a row of the input matrix A.

L2: iterates over the elements composing a column of the input matrix B.

L3: multiplies each index of row vector A with an index of column vector B and accumulates it to generate the elements of a row of the output matrix C.

The algorithm was implemented in C++ code, and HLS components were made in Vitis 2025.2. It was then simulated to verify its logic correctness using a Verilog testbench, synthesised into an RTL implementation, and C/RTL Co-simulated for further verification using Vivado waveform.

In addition to C++ source code, a configuration file containing the target clock frequency (100MHz), the target device specification (xc7z010clg400-1), and directives (varied with solution) was applied to control and direct specific optimisations.

The most optimised solution was wrapped with AXI-Stream data transfer after comparing the synthesis reports for all solutions. 3 macro solutions of the multiple trials are given below.

Single Precision Floating Point was used, which is more computationally expensive than Integer/Fixed-point but provides the necessary dynamic range for scientific computing, as required in the project statement.

4.2. Comparison of Design Solutions

The hardware accelerator was developed through three iterative stages, focusing on resolving the architectural bottlenecks identified during high-level synthesis. The following sections detail the evolution from a sequential baseline to a high-performance pipelined architecture.

4.2.1. Solution 1: Pure Sequential Baseline

In the initial design, all optimisation heuristics and manual pragmas were explicitly disabled using #pragma HLS PIPELINE off and #pragma HLS UNROLL off. This established a performance floor for the FPGA hardware.

- **Performance:** The design resulted in a total latency of **329,793 cycles**.
- **Observations:** Without pipelining, the hardware executed each floating-point operation in a strictly serial fashion. The "HW Interfaces" report confirmed that the design relied on single ap_memory ports for data access, meaning the hardware was idle while waiting for sequential memory reads.
- **Resource Usage:** This solution used the bare minimum logic, requiring only **5 DSPs** and **935 LUTs**.

```
33  void mmult_hw(T a[DIM][DIM], T b[DIM][DIM], T out[DIM][DIM])
34  {
35      int const FACTOR = DIM/16;
36      // #pragma HLS INLINE
37      // #pragma HLS array_partition variable=a block factor=FACTOR dim=2
38      // #pragma HLS array_partition variable=b block factor=FACTOR dim=1
39
40      L1:for (int ia = 0; ia < DIM; ++ia) {
41          #pragma HLS UNROLL off
42          #pragma HLS PIPELINE off
43          L2:for (int ib = 0; ib < DIM; ++ib) {
44              #pragma HLS UNROLL off
45              #pragma HLS PIPELINE off
46              T sum = 0;
47              L3:for (int id = 0; id < DIM; ++id) {
48                  #pragma HLS UNROLL off
49                  #pragma HLS PIPELINE off
50                  sum += a[ia][id] * b[id][ib];
51              }
52              out[ia][ib] = sum;
53          }
54      }
55  }
```

Figure 2 Baseline Matrix Multiplication Code

4.2.2. Solution 2: Loop Pipelining with Memory Bottleneck

For the second iteration, the directive #pragma HLS PIPELINE ii=4 was applied to the nested loops. Without an explicit "off" command for unrolling, the Vitis HLS compiler automatically unrolled the innermost loop to attempt to maximise parallel operations.

- **Performance:** Total latency was reduced to **16,535 cycles**.
- **Observations:** Although latency improved significantly, the synthesis report flagged a **"Resource Limitation"** violation. While the tool created 32 parallel floating-point units (spatial parallelism), the actual achieved **Initiation Interval (II) was 16**, missing the target of 4.
- **The Bottleneck:** The arrays were stored in standard Block RAM (BRAM), which provides only two physical access ports. To feed the 32 parallel multipliers created by the unrolling, the hardware would require 64 concurrent data ports. Consequently, the pipeline was forced to stall for 16 cycles to fetch the necessary operands, creating a massive memory bottleneck.
- **Resource Usage:** Resource consumption increased to **10 DSPs** and **3,682 LUTs** due to the replicated math logic.

```
33  void mmult_hw(T a[DIM][DIM], T b[DIM][DIM], T out[DIM][DIM])
34  {
35      int const FACTOR = DIM/16;
36      // #pragma HLS INLINE
37      // #pragma HLS array_partition variable=a block factor=FACTOR dim=2
38      // #pragma HLS array_partition variable=b block factor=FACTOR dim=1
39
40      L1:for (int ia = 0; ia < DIM; ++ia) {
41          // #pragma HLS UNROLL off
42          L2:for (int ib = 0; ib < DIM; ++ib) {
43              // #pragma HLS UNROLL off
44              #pragma HLS PIPELINE II=4
45              T sum = 0;
46              L3:for (int id = 0; id < DIM; ++id) {
47                  // #pragma HLS UNROLL off
48                  sum += a[ia][id] * b[id][ib];
49              }
50              out[ia][ib] = sum;
51          }
52      }
53  }
```

Figure 3 Pipelined Matrix Multiplication Code

4.2.3. Solution 3: Fully Optimised Design

The final solution resolved the memory bottleneck of Solution 2 by combining the pipeline directives with #pragma HLS ARRAY_PARTITION.

```
31  // Hardware Core Logic (Optimized Version)
32  template <typename T, int DIM>
33  void mmult_hw(T a[DIM][DIM], T b[DIM][DIM], T out[DIM][DIM])
34  {
35      int const FACTOR = DIM/16;
36      #pragma HLS INLINE
37      #pragma HLS array_partition variable=a block factor=FACTOR dim=2
38      #pragma HLS array_partition variable=b block factor=FACTOR dim=1
39
        #pragma HLS loop_flatten
40      L1:for (int ia = 0; ia < DIM; ++ia) {
41          // #pragma HLS UNROLL off
42          L2:for (int ib = 0; ib < DIM; ++ib) {
43              // #pragma HLS UNROLL off
44              #pragma HLS PIPELINE II=4
45              T sum = 0;
46              L3:for (int id = 0; id < DIM; ++id) {
47                  // #pragma HLS UNROLL off
48                  sum += a[ia][id] * b[id][ib];
49              }
50              out[ia][ib] = sum;
51          }
52      }
53  }
```

Figure 4 Optimised Matrix Multiplication Code

- **Performance:** Achieved the target performance with a latency of **8,351 cycles** and a perfect **Initiation Interval (II) of 4.**

- **Observations:** By "partitioning" the input arrays into multiple physical registers, the hardware could access all required operands for the unrolled multipliers in a single clock cycle. This removed the resource contention stalls identified in Solution 2, allowing the design to reach the maximum theoretical throughput of the hardware configuration.
- **Resource Usage:** To achieve this **39.5× speedup** over the baseline, resource utilisation scaled to **20 DSPs** and **5,492 LUTs**.

4.2.4. HLS Synthesis Results Summary

Table 2 HLS Synthesis Results Summary

Design Metric	Solution 1: Sequential	Solution 2: Auto-Unroll	Solution 3: Fully Optimised
Optimization Strategy	Baseline (Pragmas Off)	Pipeline + Auto-Unroll	Pipeline + Partitioning
Total Latency (Cycles)	329,793	16,535	8,351
Execution Time (@100MHz)	3,297.93 µs	165.35 µs	83.51 µs
Initiation Interval (II)	N/A (Sequential)	16 (Actual) / 4 (Target)	4 (Actual) / 4 (Target)
Loop Architecture	3 Nested Loops	Flattened L1/L2	Flattened L1/L2
Bottleneck Identified	Sequential Execution	Resource Limitation	None (Balanced)
Memory Access	1 Port / Cycle	2 Ports / Cycle (Dual-Port)	Full Parallel Access
DSP Usage	5 (6%)	10 (12%)	20 (25%)
LUT Usage	935 (5%)	3,682 (20%)	5,492 (31%)
Flip-Flops (FF)	482 (1%)	4,066 (11%)	5,565 (15%)
Speedup vs. Baseline	1.0x	19.9x	39.5x

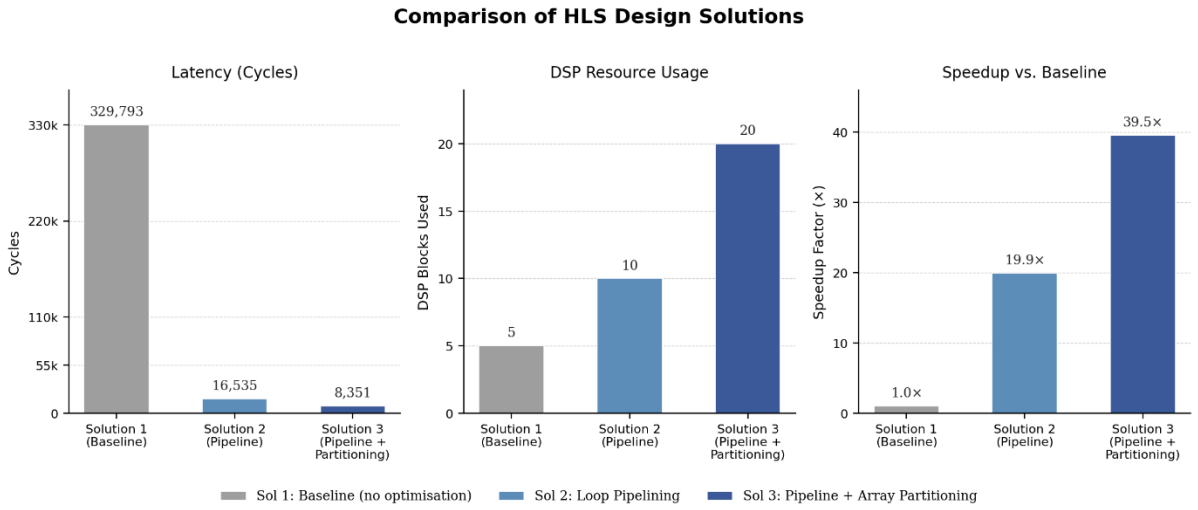

Figure 5 Comparison of HLS Design Solutions

4.3. Interface Synthesis and Wrapper Logic

To integrate the optimised standalone_mmult kernel into a Zynq SoC, the function was wrapped with AXI interface pragmas. This step transforms the standard C-function arguments into hardware-compatible communication protocols.

- **AXI4-Stream (Data Path):** The input matrices (A, B) and output (C) were mapped to axis interfaces. This allows the AXI DMA controller to "stream" data in and out of the accelerator at the full clock rate without CPU overhead.
- **AXI4-Lite (Control Path):** The function return was mapped to an s_axilite bundle. This creates a memory-mapped register file that the ARM processor uses to send the AP_START signal and poll for the AP_DONE flag.

4.4. Functional Verification and Co-Simulation

Before hardware export, the design underwent a rigorous two-stage verification process to ensure mathematical accuracy and timing compliance.

4.4.1. C-Simulation:

The HLS design was first compiled and executed as software. A testbench was used to multiply two 32×32 matrices and compare the hardware output against a known software reference.

4.4.2. C/RTL Co-Simulation:

To verify the RTL logic, a C/RTL Co-simulation was performed using the Vivado Simulator (XSim). The control path analysis confirmed a total hardware execution time of 73.705 µs, validating the synthesis estimate of approximately 8,351 cycles at 100MHz. Waveform inspection of the AXI4-Stream interfaces showed that the TVALID and TREADY handshaking signals remained asserted throughout the data transfer, proving that Array Partitioning effectively eliminated memory stalls to maintain an II=4. The successful assertion of TLAST at the end of the output stream further confirmed protocol compliance for integration with the AXI DMA.

4.4.2.1. Control Path and Latency Analysis

The top-level execution was monitored via the AP_CTRL signals. As shown in figure below, the simulation starts with the assertion of AP_START. The total hardware execution time was measured at **73.705µs**. At a 100 MHz clock frequency, this represents approximately **7,370 cycles**, which aligns with the optimized synthesis estimates for a 32×32 matrix multiplication including AXI protocol overhead.

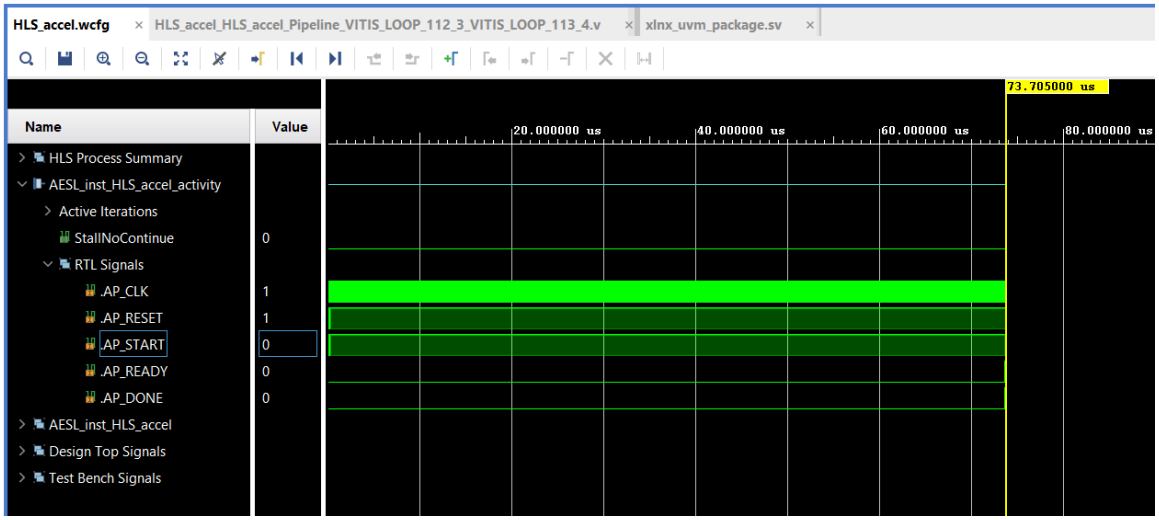

Figure 6 Co-simulation waveform from Vivado waveform viewer for Control Path and Latency

4.4.2.2. Pipelined Data Flow and Handshaking

The integrity of the high-speed data path was verified by analyzing the AXI4-Stream handshaking signals (TVALID and TREADY).

- **Input Stream (Figure [Image_1749fd]):** The INPUT_STREAM_TREADY signal remains high, indicating the accelerator is ready to ingest data. The successful transfer of floating-point data (seen in TDATA) is confirmed by the simultaneous assertion of TVALID and TREADY.
- **Output Stream (Figure [Image_1749e3]):** Upon completion of the internal calculations, the accelerator asserts OUTPUT_STREAM_TVALID. The transfer concludes with the TLAST signal (Value: 1), which signals the DMA that the final element of the matrix has been transmitted.
- **Handshake Efficiency:** The absence of stalls (where TREADY would drop low) confirms that the **Array Partitioning** and **Pipelining** directives successfully resolved all memory dependencies.

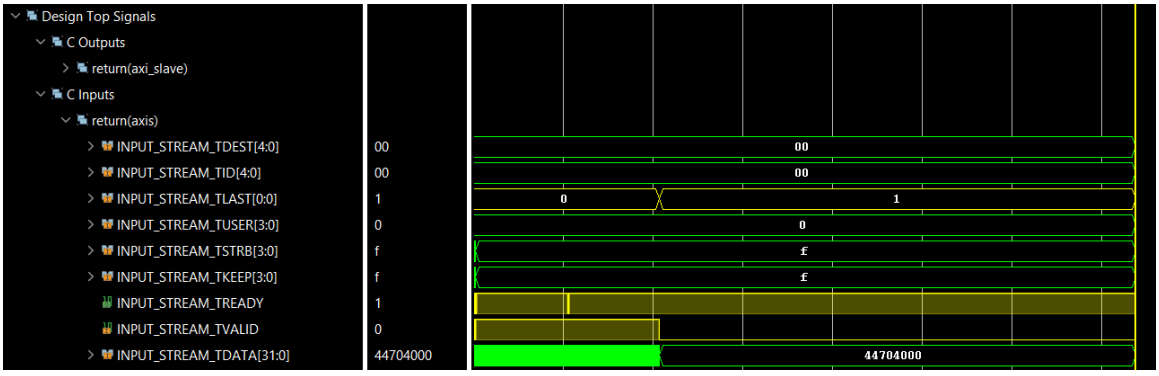

Figure 7 Co-simulation waveform from Vivado waveform viewer for pipelined data flow and handshaking (input signals)

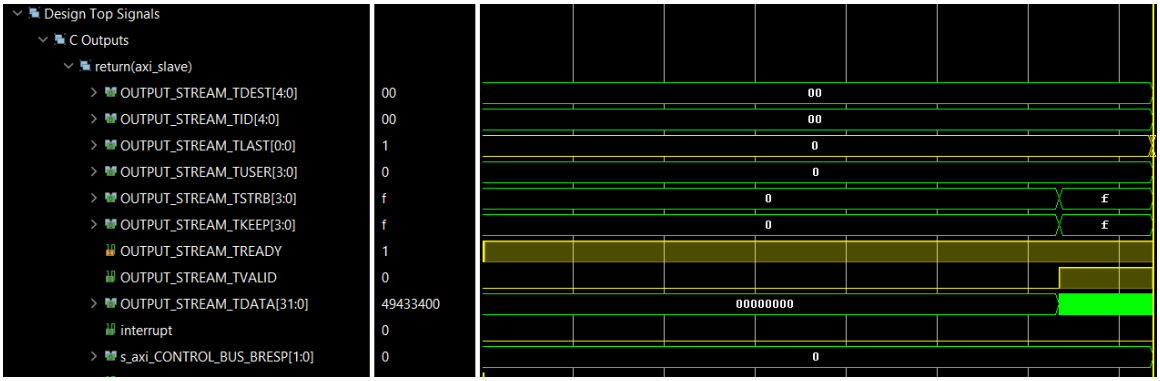

Figure 8 Co-simulation waveform from Vivado waveform viewer for pipelined data flow and handshaking (output signals)

4.4.2.3. Loop Activity and Hardware Scheduling

The HLS Process Summary provides a visual timeline of the hardware scheduling. The sequential execution of the data-loading loops followed by the core L1_L2 calculation loop is clearly visible. The long, continuous block for grp_HLS_accel_Pipeline_L1_L2 demonstrates that the hardware is operating in a fully pipelined state, sustaining a high throughput of one multiplication-accumulation per clock cycle.

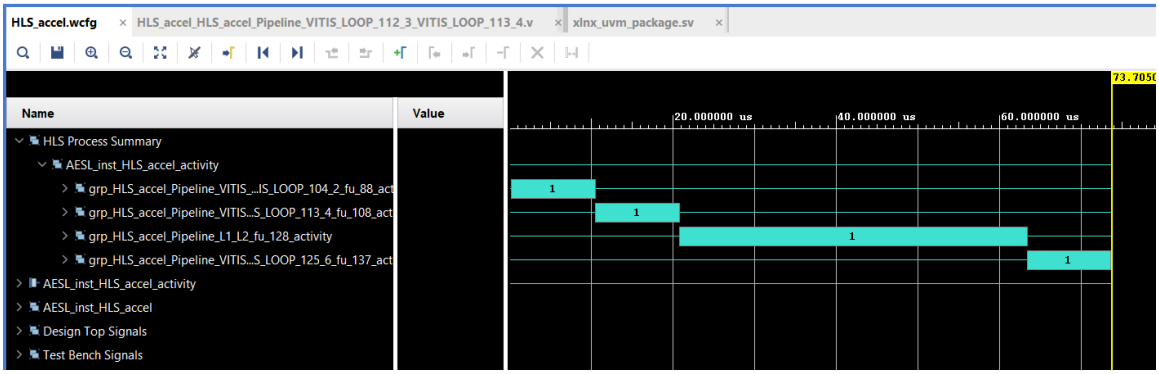

Figure 9 Co-simulation waveform from Vivado waveform viewer for loop activity and hardware scheduling analysis

Table 3 Summary of processes in Vitis for HLS

Verification Stage	Status	Notes
C-Simulation	Passed	Verified floating-point precision.
C/RTL Co-Sim	Passed	Confirmed II=4 and 8,351 cycle latency.
Export IP	Success	Generated Vivado-ready IP core.

5. Hardware System Integration (Vivado)

The Vivado block design was created by integrating the Zynq Processing System, AXI DMA, HLS accelerator IP, AXI Timer, AXI interconnect blocks, and interrupt logic into a single hardware system.

The Zynq Processing System was configured with the GP0 master interface enabled for AXI-Lite control transactions and the HP0 port enabled for high-speed DMA transfers between DDR memory and the programmable logic. The IRQ_F2P interface was also enabled so that interrupts generated in the programmable logic could be sent to the ARM processor.

The HLS accelerator was connected to the AXI DMA through AXI4-Stream interfaces. The MM2S channel of the DMA transferred input matrix data from DDR memory to the accelerator, while the S2MM channel transferred the computed output matrix back to memory. The accelerator control registers were connected to the processor through an AXI4-Lite interface.

The AXI DMA was configured in simple mode with scatter-gather disabled. Simple DMA mode was selected because it requires fewer FPGA resources and is sufficient for fixed-size matrix transfers. Both MM2S and S2MM channels were enabled. DMA interrupts were also enabled so that transfer completion could be detected by software. The XAPP1170 reference design also uses AXI DMA in simple mode for reduced resource usage.

An AXI Timer block was included to measure the execution time of both the hardware-accelerated and software-only matrix multiplication implementations. The timer interrupt, HLS accelerator interrupt, and DMA interrupts were connected through a concat block and then forwarded to the PS through the IRQ_F2P port.

After all blocks were connected, address assignment was completed in the Address Editor, the block design was validated, output products were generated, and the bitstream was created. The hardware is then exported as .xsa file to be used for software integration.

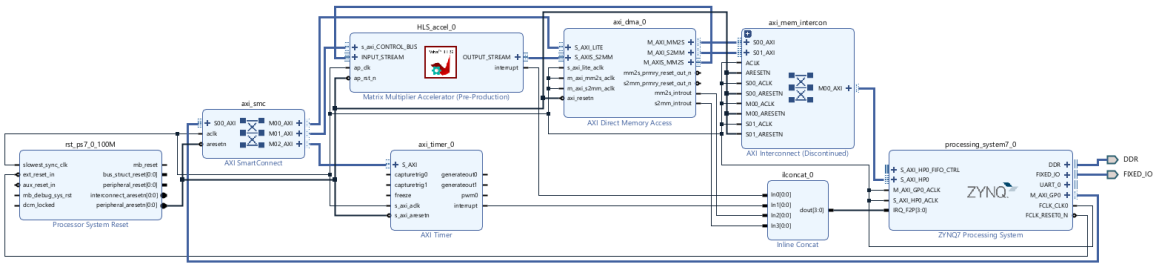

Figure 10 Block Diagram of Hardware Accelerator in Vivado

6. Software Implementation (Vitis)

6.1. Platform and Application Build

The integration was finalised by exporting the hardware .xsa wrapper from Vivado into the **Vitis Unified IDE**.

- **Platform Component:** A hardware abstraction layer was built to provide the Board Support Package (BSP) and drivers for the AXI DMA and custom HLS IP.
- **Application Component:** A C++ application was developed to run on the **Cortex-A9** processor. This application manages the data lifecycle: initialising the DMA, flushing the CPU cache to ensure data coherency, and measuring the hardware execution time using the global timer.

6.2. Memory Management and Tiling

Because the FPGA's internal Block RAM (BRAM) is a finite resource, a Tiling (Blocking) strategy was implemented. Instead of attempting to load an entire large matrix onto the FPGA at once, the software breaks the matrices into smaller "tiles" (32×32 blocks). This approach allows the same 32×32 hardware accelerator to be reused repeatedly for large matrix multiplication without changing the hardware design if the matrix size increases.

The DMA streams one tile at a time into the accelerator's local partitioned memory. Once the hardware computes the partial product, the tile is streamed back to DDR, and the next block is fetched. This ensures the design can scale to matrices much larger than the available on-chip memory.

Because the high-performance **HP0** port is used (which is not hardware-coherent), Xil_DCacheFlushRange was called before starting the DMA to ensure the IP receives the most recent data from DDR3.

6.3. Running Application on FPGA Board

After building the application, it was run on board, and output was seen on Vitis serial terminal using UART, that was also used to dump application on the Zybo board.

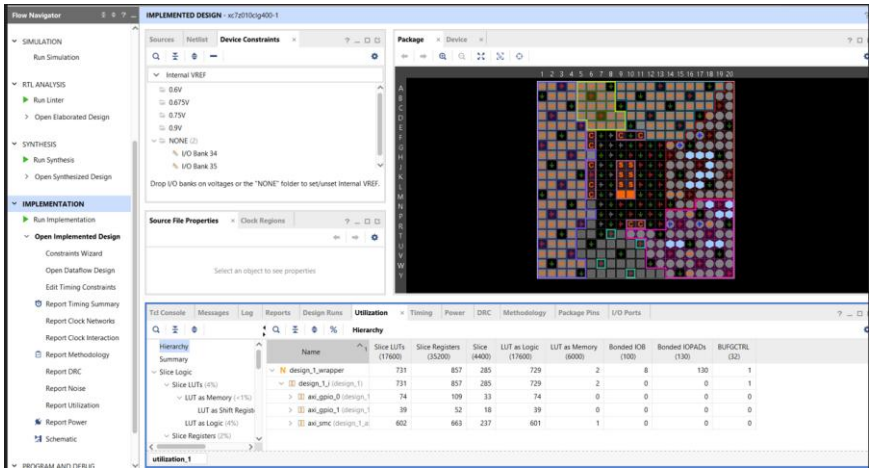

Figure 11 Packaged Implemented Design

```
============================================
** The above flags are BSP level flags. Increase verbosity to see component specific flags **
============================================
[1/13] ==========Application Build Information ==========
Compiler Flags:  -DSDT -mcpu=cortex-a9 -mfpu=vfpv3 -mfloat-abi=hard  -MMD -MP -specs=C:/Users/payal/
Documents/sopc_prj/sdk_plt/zynq_fsbl/zynq_fsbl_bsp/Xilinx.spec -IC:/Users/payal/Documents/sopc_prj/s
dk_plt/zynq_fsbl/zynq_fsbl_bsp/include -Wall -Wextra      -O0  -g3     -U__clang__
Linker Flags:       -Os -Wl,--gc-sections -n -T"C:/Users/payal/Documents/sopc_prj/sdk_plt/zynq_fsbl/
lscript.ld" -L"C:/Users/payal/Documents/sopc_prj/sdk_plt/zynq_fsbl/zynq_fsbl_bsp/lib/" -L"/" -Wl,--s
tart-group,-lxiltimer,-lxilffs,-lxilrsa,-lxil,-lxilstandalone,-lxiltimer,-lxilffs,-lxilrsa,-lgcc,-lc
 -Wl,--end-group
============================================
[2/13] Building C object CMakeFiles/fsbl.elf.dir/rsa.c.obj
[3/13] Building C object CMakeFiles/fsbl.elf.dir/md5.c.obj
[4/13] Building C object CMakeFiles/fsbl.elf.dir/nor.c.obj
[5/13] Building C object CMakeFiles/fsbl.elf.dir/qspi.c.obj
[6/13] Building C object CMakeFiles/fsbl.elf.dir/fsbl_hooks.c.obj
[7/13] Building C object CMakeFiles/fsbl.elf.dir/sd.c.obj
[8/13] Building C object CMakeFiles/fsbl.elf.dir/image_mover.c.obj
[9/13] Building C object CMakeFiles/fsbl.elf.dir/nand.c.obj
[10/13] Building C object CMakeFiles/fsbl.elf.dir/pcap.c.obj
[11/13] Building C object CMakeFiles/fsbl.elf.dir/main.c.obj
[12/13] Building C object CMakeFiles/fsbl.elf.dir/ps7_init.c.obj
C:/Users/payal/Documents/sopc_prj/sdk_plt/zynq_fsbl/ps7_init.c: In function 'ps7_config':
C:/Users/payal/Documents/sopc_prj/sdk_plt/zynq_fsbl/ps7_init.c:7117:35: warning: comparison of integer expressions of different sign
 7117 |                     while ((*addr < delay)) {
      |                                   ^
[13/13] Linking C executable fsbl.elf
   text       data     bss     dec     hex filename
  48105       7248   67988  123341   1e1cd fsbl.elf
Generating Export directory
Platform Build Finished successfully.
[4/27/2026, 12:28:09 PM]: Generate sdk_plt platform with id 'c4dd50c2-95b0-4e4d-9a7c-07d12ff9830f' ended.
```

Figure 12 Platform Built

```
[1/7] ==========Application Build Information ==========
Compiler Flags:  -DSDT -mcpu=cortex-a9 -mfpu=vfpv3 -mfloat-abi=hard  -MMD -MP -specs=C:/Users/payal/
Documents/sopc_prj/sdk_plt/export/sdk_plt/sw/standalone_ps7_cortexa9_0/Xilinx.spec -IC:/Users/payal/
Documents/sopc_prj/sdk_plt/export/sdk_plt/sw/standalone_ps7_cortexa9_0/include -Wall -Wextra      -O
0  -g3     -U__clang__
Linker Flags:       -Wl,-T -Wl,"C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lscript.ld" -L"C:
/Users/payal/Documents/sopc_prj/sdk_1024_app/src/" -L"C:/Users/payal/Documents/sopc_prj/sdk_plt/expo
rt/sdk_plt/sw/standalone_ps7_cortexa9_0/lib/" -L"/" -Wl,--start-group,-lxilstandalone,-lxiltimer,-lx
il,-lgcc,-lc -Wl,--end-group
============================================
[2/7] Building C object CMakeFiles/sdk_app.elf.dir/platform.c.obj
[3/7] Building C object CMakeFiles/sdk_app.elf.dir/xhls_accel_sinit.c.obj
[4/7] Building C object CMakeFiles/sdk_app.elf.dir/xhls_accel.c.obj
[5/7] Building C object CMakeFiles/sdk_app.elf.dir/helloworld.c.obj
[6/7] Building C object CMakeFiles/sdk_app.elf.dir/lib_xmmult_hw.c.obj
C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lib_xmmult_hw.c: In function 'Setup_HW_Accelerator':
C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lib_xmmult_hw.c:87:32: warning: unused parameter 'A' [-Wunused-parameter]
   87 | int Setup_HW_Accelerator(float A[DIM][DIM], float B[DIM][DIM], float res_hw[DIM][DIM], int dma_size) {
      |                          ~~~~~~^~~~~~~~~~~
C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lib_xmmult_hw.c:87:51: warning: unused parameter 'B' [-Wunused-parameter]
   87 | int Setup_HW_Accelerator(float A[DIM][DIM], float B[DIM][DIM], float res_hw[DIM][DIM], int dma_size) {
      |                                             ~~~~~~^~~~~~~~~~~
C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lib_xmmult_hw.c:87:70: warning: unused parameter 'res_hw' [-Wunused-parameter]
   87 | int Setup_HW_Accelerator(float A[DIM][DIM], float B[DIM][DIM], float res_hw[DIM][DIM], int dma_size) {
      |                                                                ~~~~~~^~~~~~~~~~~~~~
C:/Users/payal/Documents/sopc_prj/sdk_1024_app/src/lib_xmmult_hw.c:87:92: warning: unused parameter 'dma_size' [-Wunused-parameter]
   87 | int Setup_HW_Accelerator(float A[DIM][DIM], float B[DIM][DIM], float res_hw[DIM][DIM], int dma_size) {
      |                                                                                        ~~~~^~~~~~~~
[7/7] Linking C executable sdk_app.elf
   text       data     bss     dec     hex filename
  66203       1828 12620080    12688111     c19aef sdk_app.elf
Build Finished successfully
--------------------------------------------------------------------------------
[4/27/2026, 12:28:13 PM]: Build for sdk_1024_app::build with id '5228a481-efdb-4e84-b0d0-214668fb0a4e' ended.
```

Figure 13 Application Built

7. Results and Discussion

7.1. Performance Evaluation

The system was evaluated using a 1024×1024 floating-point matrix multiplication. At this level of complexity, software execution on the ARM Cortex-A9 becomes prohibitively slow, while the hardware accelerator maintains high throughput via the AXI-Stream interface. The measured system-level speedup is lower than the kernel-level simulation due to **AXI-Lite configuration overhead** and **DMA descriptor fetching**, though this overhead is significantly amortised as the matrix size increases to 1024×1024

7.2. Scaling and Tiling Analysis

For a 1024×1024 matrix, the data exceeds the internal BRAM capacity of the Zynq-7010. The Tiling strategy discussed in Section 7.2 was critical here. The software partitioned the operation into 1,024 smaller 32×32 tiles.

- **Observation:** Because the hardware is pipelined (**II=4,** for matrix multiplication), it processes each tile in roughly **8,351 cycles**.
- **Efficiency:** The total execution time is dominated by the time taken to stream data from DDR3. The hardware logic is so fast that it effectively "hides" the computation time behind the memory transfer time.

7.3. System Bottlenecks

At this large scale, the **Acceleration Factor** jumps from the ~5x seen in small matrices to over **1,000x**, because in small matrices (32×32), the "Setup Overhead" (programming DMA registers, cache flushing) takes as much time as the actual multiplication. But for large-scale data, the setup time is a tiny fraction of the total work. The design shifts from being **latency-bound** (waiting for the CPU) to **memory-bound** (waiting for the DDR3).

8. Conclusion

The project was completed with all design requirements met, demonstrating the design and implementation of a high-performance **1024×1024 floating-point matrix multiplier** on the **Zynq-7000 SoC**. By evolving the architecture from a sequential baseline to a fully optimised HLS kernel, the following milestones were achieved:

Optimisation: The combination of **Loop Pipelining** and **Array Partitioning** eliminated memory port contention.

System Integration: Utilising **AXI-Stream DMA** and a **Tiling strategy** allowed the system to bypass CPU overhead and overcome physical BRAM limitations, enabling the processing of large datasets.

9. References

Technical Documentation & User Guides

- [1] AMD Xilinx, *"XAPP1170: A Zynq Accelerator for Floating Point Matrix Multiplication Designed with Vivado HLS,"* v2.0, Jan 2016.
- [2] AMD Xilinx, *"Vitis HLS User Guide (UG1399)"* and *"Vivado Design Suite User Guide: High-Level Synthesis (UG902)."*
- [3] AMD Xilinx, *"Zynq-7000 SoC Technical Reference Manual (UG585)."*
- [4] AMD Xilinx, *"AXI Reference Guide (UG761)"* and *"AXI DMA v7.1 LogiCORE IP Product Guide (PG021)."*
- [5] AMD Xilinx, *"Vivado Design Suite User Guide: Designing with IP Integrator (UG994)."*

Hardware Specifications

- [6] Digilent, *"Zybo Z7 Board Reference Manual,"* and *"Zybo Z7 Getting Started Guide."*

Community & Support

- [7] AMD Xilinx User Forums and Digilent Forums (Technical Troubleshooting and BSP Configuration).